

vLLM V1 – how one mixed batch becomes FlashAttention inputs

Toy batch: R0 = chunked prefill (2nd chunk), R1 = decode, R2 = new prefill. block_size = 4 (illustrative), M = max_model_len. Source: vllm @ 4c2d2776; GR = v1/worker/gpu_model_runner.py, BT = v1/worker/block_table.py, FA = v1/attention/backends/flash_attn.py

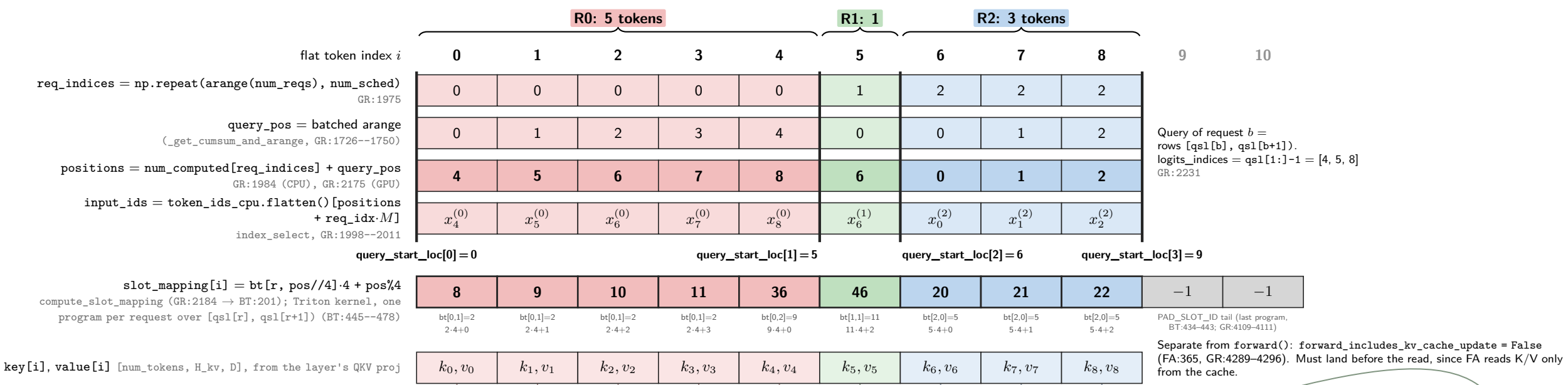
(1) Per-request inputs: scheduler output + persistent InputBatch

req (batch row)	phase	num_computed_tokens	num_scheduled_tokens	block_table row
R0 (row 0)	chunked prefill, prompt 10, chunk 2	4	5	[7, 2, 9]
R1 (row 1)	decode (1 new token)	6	1	[4, 11]
R2 (row 2)	new prefill, prompt 3	0	3	[5]

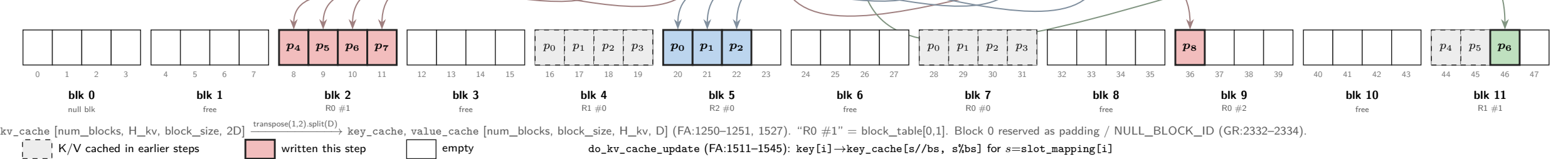
num_computed_tokens_cpu (InputBatch); num_scheduled_tokens_np from scheduler_output (GR:4223--4227)
block_table: BlockTable.block_table [max_num_reqs, max_blocks] int32 (BT:114), filled by append_row (BT:157); H2D by commit_block_table (GR:1971)

Per-request attention metadata (_prepare_inputs, GR:1951)
query_start_loc = [0]||cumsum(num_sched) = [0, 5, 6, 9]
GR:2060--2066; tail padded with 9 (non-decreasing for FA)
seq_lens = num_computed + num_sched = [9, 7, 3] GR:2179--2182
max_query_len = max(num_sched) = 5 GR:4227 → 4365
max_seq_len = max(optimistic_seq_lens_cpu) = 9 GR:2315
block_table = block_table.gpu[:num_reqs] GR:2329--2330, BT:269
→ CommonAttentionMetadata(..., causal=True) GR:2426--2443 →
FlashAttentionMetadata FA:996--1019

(2) Flatten scheduled tokens into one token axis (_prepare_inputs, GR:1951–2188)

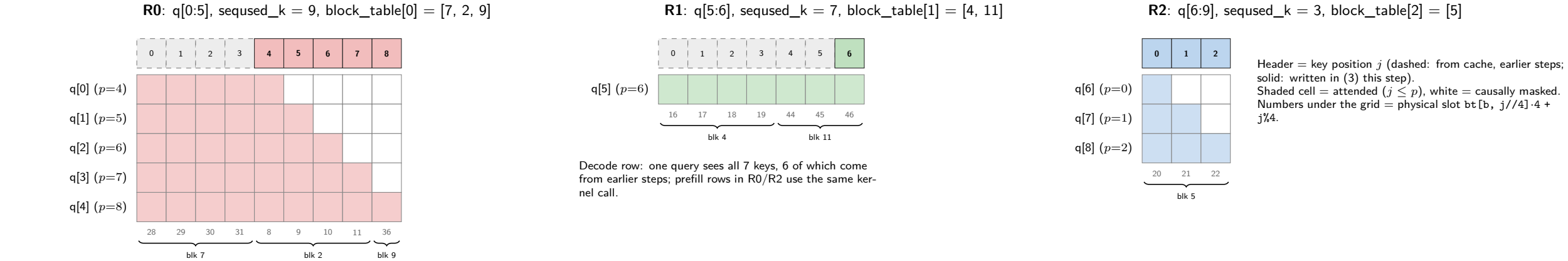


(3) WRITE: reshape_and_cache_flash



(4) READ: one varlen FlashAttention call over the whole mixed batch (FlashAttentionImpl.forward, FA:1231)

flash_attn_varlen_func(q=query[:num_actual_tokens], k=key_cache, v=value_cache, cu_seqlens_q=query_start_loc, seqlen_k=seq_len_k, max_seqlen_q=max_query_len, max_seqlen_k=max_seq_len, block_table=block_table, causal=True, ...) FA:1330--1335, 1454--1479 (_FA4_DENSE_ATTENTION_KERNEL → flash_attn_varlen_func, FA:116)
For request b : queries = q[qs1[b]:qs1[b+1]]; keys/values = positions 0..seq_lens[b]-1 gathered page by page via block_table[b, j//bs]; causal mask aligned to the end of the sequence, so the query at position p sees keys $j \leq p$.



Assumptions: dcp_world_size = 1 (BT:448 reduces to pos // block_size), kernel block = KV block (blocks_per_kv_block = 1), no cascade (common_prefix_len = 0, FA:871), no spec decode, no CUDA-graph padding of requests. block_size = 4 and the block ids are for illustration only (FA needs a multiple of 16, FA:363). Batch rows are kept in scheduler order; FA handles prefill and decode rows in the same call.